Reconsidering the `std::execution::on` algorithm

`on` *second thought*

Authors:	Eric Niebler
Date:	May 14, 2024
Source:	GitHub
Issue tracking:	GitHub
Project:	ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++
Audience:	LEWG

Synopsis

Usage experience with P2300 has revealed a gap between users' expectations and the actual behavior of the `std::execution::on` algorithm. This paper seeks to close that gap by making its behavior less surprising.

Executive Summary

Below are the specific changes this paper proposes:

1. Rename the current `std::execution::on` algorithm to `std::execution::start_on`.
2. Rename `std::execution::transfer` to `std::execution::continue_on`
3. Optional: Add a new algorithm `std::execution::on` that, like `start_on`, starts a sender on a particular context, but that remembers where execution is transitioning *from*. After the sender completes, the `on` algorithm transitions *back* to the starting execution context, giving a scoped, there-and-back-again behavior.
4. Optional: Add a form of `execution::on` that lets you run part of a *continuation* on one scheduler, automatically transitioning back to the starting context.

Revisions

- **R2:**
  - Give the `on` algorithm an unspecified tag type to discourage its customization. See discussion in On the customizability of the new `execution::on` algorithm.
  - Place strict constraints on `on` customizations so that they have the correct semantics.
  - Adds a discussion about naming for the `start_on` and `continue_on` algorithms. See the discussion in On the naming of `start_on` and `continue_on`.
- **R1:**

- Makes the `write_env` adaptor exposition-only, removes the `finally` and `unstoppable` adaptors, and reverts the changes to `schedule_from` and the `let_` algorithms.
- A follow-on paper, P3284, will propose to add `write_env`, `unstoppable`, and `finally` as proper members of the Standard Library.

- **R0:**
  - Initial revision

Problem Description

If, knowing little about senders and sender algorithms, someone showed you code such as the following:

```
namespace ex = std::execution;

ex::sender auto work1 = ex::just()
                      | ex::transfer(scheduler_A);

ex::sender auto work2 = ex::on(scheduler_B, std::move(work1))
                      | ex::then([] { std::puts("hello world!"); });

ex::sender auto work3 = ex::on(scheduler_C, std::move(work2))

std::this_thread::sync_wait(std::move(work3));
```

… and asked you, which scheduler, `scheduler_A` or `scheduler_B`, is used to execute the code that prints `"hello world!"`? You might reasonably think the answer is `scheduler_C`. Your reasoning would go something like this:

> Well clearly the first thing we execute is `on(scheduler_C, work2)`. I'm pretty sure that is going to execute `work2` on `scheduler_C`. The `printf` is a part of `work2`, so I'm going to guess that it executes on `scheduler_C`.

This paper exists because the `on` algorithm as specified in P2300R8 does *not* print `"hello world!"` from `scheduler_C`. It prints it from `scheduler_A`. Surprise!

But why?

`work2` executes `work1` on `scheduler_B`. `work1` then rather rudely transitions to `scheduler_A` and doesn't transition back. The `on` algorithm is cool with that. It just happily runs its continuation inline, *still on `scheduler_A`*, which is where `"hello world!"` is printed from.

If there was more work tacked onto the end of `work3`, it too would execute on `scheduler_A`.

User expectations

The authors of P2300 have witnessed this confusion in the wild. And when this author has asked his programmer friends about the code above, every single one said they expected behavior different from what is specified. This is very concerning.

However, if we change some of the algorithm names, people are less likely to make faulty assumptions about their behavior. Consider the above code with different names:

```
namespace ex = std::execution;

ex::sender auto work1 = ex::just()
                      | ex::continue_on(scheduler_A);
```

```
ex::sender auto work2 = ex::start_on(scheduler_B, std::move(work1))
                      | ex::then([] { std::puts("hello world!"); });

ex::sender auto work3 = ex::start_on(scheduler_C, std::move(work2))

std::this_thread::sync_wait(std::move(work3));
```

Now the behavior is a little more clear. The names `start_on` and `continue_on` both suggest a one-way execution context transition, which matches their specified behavior.

Filling the gap

`on` fooled people into thinking it was a there-and-back-again algorithm. We propose to fix that by renaming it to `start_on`. But what of the people who *want* a there-and-back-again algorithm?

Asynchronous work is better encapsulated when it completes on the same execution context that it started on. People are surprised, and reasonably so, if they `co_await` a task from a CPU thread pool and get resumed on, say, an OS timer thread. Yikes!

We have an opportunity to give the users of P2300 what they *thought* they were already getting, and now the right name is available: **`on`**.

We propose to add a new algorithm, called `on`, that remembers where execution came from and automatically transitions back there. Its operational semantics can be easily expressed in terms of the existing P2300 algorithms. It is approximately the following:

```
template <ex::scheduler Sched, ex::sender Sndr>
sender auto on(Sched sch, Sndr sndr) {
  return ex::read(ex::get_scheduler)
       | ex::let_value([=](auto orig_sch) {
           return ex::start_on(sch, sndr)
                | ex::continue_on(orig_sch);
         });
}
```

One step further?

Once we recast `on` as a there-and-back-again algorithm, it opens up the possibility of another there-and-back-again algorithm, one that executes a part of a *continuation* on a given scheduler. Consider the following code, where `async_read_file` and `async_write_file` are functions that return senders (description after the break):

```
ex::sender auto work = async_read_file()
                     | ex::on(cpu_pool, ex::then(crunch_numbers))
                     | ex::let_value([](auto numbers) {
                         return async_write_file(numbers);
                       });
```

Here, we read a file and then send it to an `on` sender. This would be a different overload of `on`, one that takes a sender, a scheduler, and a continuation. It saves the result of the sender, transitions to the given scheduler, and then forwards the results to the continuation, `then(crunch_numbers)`. After that, it returns to the previous execution context where it executes the `async_write_file(numbers)` sender.

The above would be roughly equivalent to:

```
ex::sender auto work = async_read_file()
                     | ex::let_value([=](auto numbers) {
                         ex::sender auto work = ex::just(numbers)
                                              | ex::then(crunch_numbers);
```

```
    return ex::on(cpu_pool, work)
         | ex::let_value([=](auto numbers) {
             return async_write_file(numbers);
           });
});
```

This form of `on` would make it easy to, in the middle of a pipeline, pop over to another execution context to do a bit of work and then automatically pop back when it is done.

Implementation Experience

The perennial question: has it been implemented? It has been implemented in stdexec for over a year, modulo the fact that `stdexec::on` has the behavior as specified in P2300R8, and a new algorithm `exec::on` has the there-and-back-again behavior proposed in this paper.

Design Considerations

Do we really have to rename the `transfer` algorithm?

We don't! Within sender expressions, `work | transfer(over_there)` reads a bit nicer than `work | continue_on(over_there)`, and taken in isolation the name change is strictly for the worse.

However, the symmetry of the three operations:

- `start_on`
- `continue_on`
- `on`

… encourages developers to infer their semantics correctly. The first two are one-way transitions before and after a piece of work, respectively; the third book-ends work with transitions. In the author's opinion, this consideration outweighs the other.

Do we need the additional form of `on`?

We don't! Users can build it themselves from the other pieces of P2300 that will ship in C++26. But the extra overload makes it much simpler for developers to write well-behaved asynchronous operations that complete on the same execution contexts they started on, which is why it is included here.

What happens if there's no scheduler for `on` to go back to?

If we recast `on` as a there-and-back-again algorithm, the implication is that the receiver that gets `connect`-ed to the `on` sender must know the current scheduler. If it doesn't, the code will not compile because there is no scheduler to go back to.

Passing an `on` sender to `sync_wait` will work because `sync_wait` provides a `run_loop` scheduler as the current scheduler. But what about algorithms like `start_detached` and `spawn` from P3149? Those algorithms connect the input sender with a receiver whose environment lacks a value for the `get_scheduler` query. As specified in this paper, those algorithms will reject `on` senders, which is bad from a usability point of view.

There are a number of possible solutions to this problem:

1. Any algorithm that eagerly `connect`s a sender should take an environment as an optional extra argument. That way, users have a way to tell the algorithm what the current scheduler is. They can also pass additional

information like allocators and stop tokens. **UPDATE:** On 2024-05-21, straw polling indicated that LEWG would like to see a paper proposing this.

2. Those algorithms can specify a so-called "inline" scheduler as the current scheduler, essentially causing the `on` sender to perform a no-op transition when it completes. **UPDATE:** On 2024-05-21, LEWG opted to not pursue this option.

3. Those algorithms can treat top-level `on` senders specially by converting them to `start_on` senders. **UPDATE:** On 2024-05-21, LEWG opted to not pursue this option.

4. Those algorithms can set a hidden, non-forwarding "root" query in the environment. The `on` algorithm can test for this query and, if found, perform a no-op transition when it completes. This has the advantage of not setting a "current" scheduler, which could interfere with the behavior of nested senders. **UPDATE:** On 2024-05-21, LEWG opted to not pursue this option.

Questions for LEWG's consideration

The author would like LEWG's feedback on the following two questions:

1. If `on` is renamed `start_on`, do we also want to rename `transfer` to `continue_on`? **UPDATE:** On 2024-05-13, LEWG straw polling answered this question in the affirmative.

2. If `on` is renamed `start_on`, do we want to add a new algorithm named `on` that book-ends a piece of work with transitions to and from a scheduler? **UPDATE:** On 2024-05-13, LEWG straw polling answered this question in the affirmative.

3. If we want the new scoped form of `on`, do we want to add the `on(sndr, sch, continuation)` algorithm overload to permit scoped execution of continuations? **UPDATE:** On 2024-05-13, LEWG straw polling answered this question in the affirmative.

On the customizability of the new `execution::on` algorithm

On the 2024-05-21 telecon, LEWG requested to see a revision of this paper that removes the customizability of the proposed `execution::on` algorithm. The author agrees with this guidance in principle: the behavior of `on` should be expressed in terms of `start_on` and `continue_on`, and users should be customizing those instead.

However, the author now realizes that to ban customization of `on` would make it impossible to write a recursive sender tree transformation without intrusive design changes to P2300. Consider that the author of an execution domain `D` might want a transformation to be applied to every sender in an expression tree. They would like for this expression:

```
std::execution::transform_sender(D(), std::execution::on(sch, child), env);
```

to be equivalent to:

```
std::execution::on(sch, std::execution::transform_sender(D(), child, env));
```

The ability to crack open a sender, transform the children, and reassemble the sender is essential for these sorts of recursive transformations, but that ability *also* permits other, more general transformations. The author strongly feels that disallowing transformations of `on` would be a step in the wrong direction.

However, there are a few things we can do to discourage users from customizing `on` in ways we disapprove.

1. Give the `on` algorithm an unspecified tag type so that it is a little awkward to single the `on` algorithm out for special treatment by a domain's `transform_sender`.

2. Place strict requirements on customizations of `on` to ensure correct program semantics in the presence of customizations. Violations of these requirements would lead to undefined behavior.

These changes have been applied as of revision 2 of this paper.

On the naming of `start_on` and `continue_on`

It was pointed out in the 2024-05-14 LEWG telecon, and again on 2024-05-21, that the name `start_on` is potentially confusing given that "`start`" in P2300 means "start *now*." The `start_on` algorithms does not mean "start now"; it means, "*when* the work is started, start it *there*."

The authors of P2300 make the following suggestion:

- Rename `start_on` to `starts_on`.
- Rename `continue_on` to `continues_on`.

The naming of these algorithms should be determined by LEWG the next time this paper is discussed.

Proposed Wording

[Editorial note: The wording in this section is based on P2300R9 with the addition of P8255R1. -- *end note]*

[Editorial note: Change [exec.syn] as follows: -- *end note]*

```
...
  struct start_on_t;
  struct transfer_tcontinue_on_t;
  struct schedule_from_t;
...
  inline constexpr start_on_t start_on{};
  inline constexpr transfer_t transfercontinue_on_t continue_on{};
  inline constexpr unspecified on{};
  inline constexpr schedule_from_t schedule_from{};
```

[Editorial note: Add a new paragraph (15) to section [exec.snd.general], paragraph 3 as follows: -- *end note]*

15. `template<sender Sndr, queryable Env>`
    `constexpr auto write-env(Sndr&& sndr, Env&& env); // exposition only`

    1. `write-env` is an exposition-only sender adaptor that, when connected with a receiver `rcvr`, connects the adapted sender with a receiver whose execution environment is the result of joining the `queryable` argument `env` to the result of `get_env(rcvr)`.

    2. Let `write-env-t` be an exposition-only empty class type.

    3. *Returns:* `make-sender(make-env-t(), std::forward<Env>(env), std::forward<Sndr>(sndr))`.

    4. *Remarks:* The exposition-only class template `impls-for` ([exec.snd.general]) is specialized for `write-env-t` as follows:

```
template<>
struct impls-for<write-env-t> : default-impls {
  static constexpr auto get-env =
    [](auto, const auto& state, const auto& rcvr) noexcept {
```

```
      return JOIN-ENV(state, get_env(rcvr));
    };
};
```

[Editorial note: Change subsection "`execution::on` [exec.on]" to "`execution::start_on` [exec.start.on]", and within that subsection, replace every instance of "`on`" with "`start_on`" and every instance of "`on_t`" with "`start_on_t`". -- *end note]*

[Editorial note: Change subsection "`execution::transfer` [exec.transfer]" to "`execution::continue_on` [exec.complete.on]", and within that subsection, replace every instance of "`transfer`" with "`continue_on`" and every instance of "`transfer_t`" with "`continue_on_t`". -- *end note]*

[Editorial note: Insert a new subsection "`execution::on` [exec.on]" as follows: -- *end note]*

`execution::on` [exec.on]

1. The `on` sender adaptor has two forms:
   - one that starts a sender `sndr` on an execution agent belonging to a particular scheduler's associated execution resource and that restores execution to the starting execution resource when the sender completes, and
   - one that, upon completion of a sender `sndr`, transfers execution to an execution agent belonging to a particular scheduler's associated execution resource, then executes a sender adaptor closure with the async results of the sender, and that then transfers execution back to the execution resource `sndr` completed on.
2. The name `on` denotes a customization point object. For some subexpressions `sch` and `sndr`, if `decltype((sch))` does not satisfy `scheduler`, or `decltype((sndr))` does not satisfy `sender`, `on(sch, sndr)` is ill-formed.
3. Otherwise, the expression `on(sch, sndr)` is expression-equivalent to:

```
transform_sender(
  query-or-default(get_domain, sch, default_domain()),
  make-sender(on, sch, sndr));
```

4. For a subexpression `closure`, if `decltype((closure))` is not a sender adaptor closure object ([exec.adapt.objects]), the expression `on(sndr, sch, closure)` is ill-formed; otherwise, it is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(on, pair{sch, closure}, sndr));
```

5. Let `out_sndr` and `env` be subexpressions, let `OutSndr` be `decltype((out_sndr))`, and let `Env` be `decltype((env))`. If *sender-for*<OutSndr, on_t> is `false`, then the expressions `on.transform_env(out_sndr, env)` and `on.transform_sender(out_sndr, env)` are ill-formed; otherwise:
   1. Let *none-such* be an unspecified empty class type, and let *not-a-sender* be the exposition-only type:

```
struct not-a-sender {
  using sender_concept = sender_t;

  auto get_completion_signatures(auto&&) const {
    return see below;
```

```
  }
};
```

… where the member function `get_completion_signatures` returns an object of a type that is not a specialization of the `completion_signatures` class template.

2. `on.transform_env(out_sndr, env)` is equivalent to:

```
auto&& [ign1, data, ign2] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
  return JOIN-ENV(SCHED-ENV(data), FWD-ENV(std::forward<Env>(env)));
} else {
  return std::forward<Env>(env);
}
```

3. `on.transform_sender(out_sndr, env)` is equivalent to:

```
auto&& [ign, data, sndr] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
  auto orig_sch =
    query-with-default(get_scheduler, env, none-such());

  if constexpr (same_as<decltype(orig_sch), none-such>) {
    return not-a-sender{};
  } else {
    return continue_on(
      start_on(std::forward_like<OutSndr>(data), std::forward_like<OutSndr>(sndr)),
      std::move(orig_sch));
  }
} else {
  auto&& [sch, closure] = std::forward_like<OutSndr>(data);
  auto orig_sch = query-with-default(
    get_completion_scheduler<set_value_t>,
    get_env(sndr),
    query-with-default(get_scheduler, env, none-such()));

  if constexpr (same_as<decltype(orig_sch), none-such>) {
    return not-a-sender{};
  } else {
    return write-env(
      continue_on(
        std::forward_like<OutSndr>(closure)(
          continue_on(
            write-env(std::forward_like<OutSndr>(sndr), SCHED-ENV(orig_sch)),
            sch)),
        orig_sch),
      SCHED-ENV(sch));
  }
}
```

4. *Recommended practice:* Implementations should use the return type of *`not-a-sender`*`::get_completion_signatures` to inform users that their usage of `on` is incorrect because there is no available scheduler onto which to restore execution.

[Editorial note: The following two paragraphs are new in R2. -- *end note]*

6. Let the subexpression `out_sndr` denote the result of the invocation `on(sch, sndr)` or an object copied or moved from such, let `OutSndr` be `decltype((out_sndr))`, let the subexpression `rcvr` denote a receiver such that `sender_to<decltype((out_sndr)), decltype((rcvr))>` is `true`, and let `sch_copy` and `sndr_copy` be lvalue subexpressions refering to objects decay-copied from `sch` and `sndr` respectively.

   The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation as if by calling `connect(S, rcvr)`, where `S` is a sender expression semantically equivalent

to:

```
continue_on(
  start_on(std::forward_like<OutSndr>(sch_copy), std::forward_like<OutSndr>(sndr_copy)),
  orig_sch)
```

where `orig_sch` is `get_scheduler(rcvr)`.

7. Let the subexpression `out_sndr2` denote the result of the invocation `on(sndr, sch, closure)` or an object copied or moved from such, let `OutSndr2` be `decltype((out_sndr2))`, let the subexpression `rcvr2` denote a receiver such that `sender_to<decltype((out_sndr2)), decltype((rcvr2))>` is `true`, and let `sndr_copy`, `sch_copy`, and `closure_copy` be lvalue subexpressions refering to objects decay-copied from `sndr`, `sch`, and `closure` respectively.

   The expression `connect(out_sndr2, rcvr2)` has undefined behavior unless it creates an asynchronous operation as if by calling `connect(S2, rcvr2)`, where `S2` is a sender expression semantically equivalent to:

```
write-env(
  continue_on(
    std::forward_like<OutSndr2>(closure_copy)(
      continue_on(
        write-env(std::forward_like<OutSndr2>(sndr_copy), SCHED-ENV(orig_sch)),
        sch_copy)),
    orig_sch),
  SCHED-ENV(sch_copy))
```

   where `orig_sch` is an lvalue refering to an object decay-copied from `get_completion_scheduler<set_value_t>(get_env(sndr_copy))` if that expression is well-formed; otherwise, `get_scheduler(get_env(rcvr2))`.

Acknowlegments

I'd like to thank my dog, Luna.